59

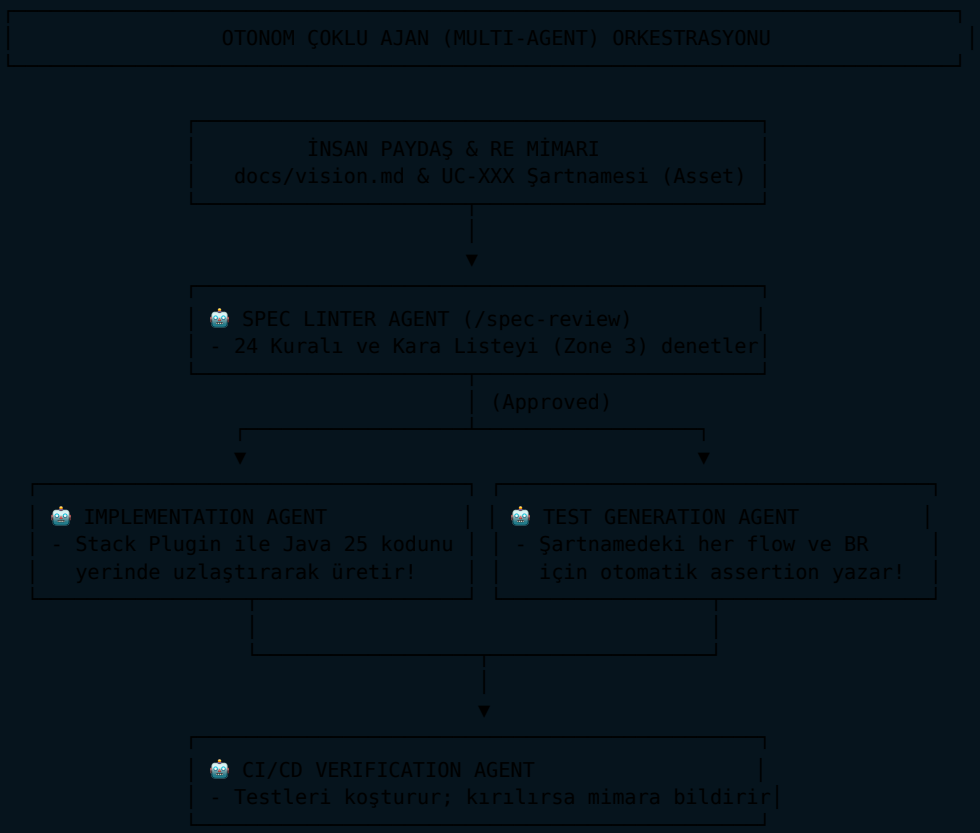
BÖLÜM: KONTROL LISTELERI, YAYGIN HATALAR VE MAANG MIMAR SEVIYESI USTALIK | MODULE: CHECKLISTS, COMMON MISTAKES & MAANG ARCHITECT MASTERY

# Otonom Çoklu Ajan (Multi-Agent) İş Akışları ve Şartname Tabanlı Kod Üretimi

Autonomous Multi-Agent AI Workflows: Orchestrating Spec-Driven Code Generation

## ⚡ Akış, Varlık & Süreç Şeması

## ⚡ Process & Artifact Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Bir inşaat şantiyesindeki robot işçileri düşünün: Mimar (İnsan) elindeki kesin mimari planı (Use Case Şartnamesi) robot şefe verir. Robot şef planı kontrol eder. Sonra bir robot duvarları örerken (Kod Ajanı), diğer robot betonun sağlamlığını lazerle ölçer (Test Ajanı). Kimse kafasına göre bina yapmaz; herkes mimarın tek bir çizimine bakar!

### 🇬🇧 ENGLISH (ELI5)

Imagine a high-tech robotic construction site: The human architect provides the structural CAD blueprint (The Use Case Spec). A supervisor robot audits the blueprint. Then, while one robotic arm lays the precision steel beams (Coding Agent), a companion drone fires lasers to verify structural integrity (Testing Agent). All autonomous agents align around a single blueprint!

## 🔍 MAANG & AIUP Seviyesi Teknik Derinlik

### 🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Otonom Çoklu Ajan Orkestrasyonu (Multi-Agent Workflows): Yapay zekâ yazılım mühendisliği artık tek bir 'Chat GPT' penceresine prompt yazmaktan çıkmış, uzmanlaşmış çoklu ajanların (Multi-Agent Systems) işbirliğine dönüşmüştür. Ancak kontrolsüz ajanlar birbirlerinin kodunu bozabilir ve halüsinasyon sarmalına girebilir. AI Unified Process bu sorunu şartnameyi 'ortak koordinasyon düzlemi' (shared coordination substrate) yaparak çözer: 1) Spec Linter Ajanı: Şartnamenin sözdizimini, 24 kuralını ve Zone 3 temizliğini denetler. 2) Implementasyon Ajanı: Şartnameden sadece onaylanmış adımları okuyarak Java kodunu üretir. 3) Test Üretim Ajanı: Doğrulama adımlarını ve BR kurallarını bağımsız olarak test koduna dönüştürür. Kod üreten ajan ile test üreten ajanın birbirlerinden habersiz olarak sadece ortak şartnameyi okuması, 'kendi yazdığı kodu test etme' önyargısını tamamen ortadan kaldırır. 4) Doğrulama Ajanı: Kod ve testleri derleyip CI/CD ortamında koşturur.

## 🔍 MAANG & AIUP Technical Deep-Dive

### 🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Orchestrating Spec-Driven Multi-Agent Workflows: Autonomous AI software development has outgrown monolithic chat interfaces in favor of specialized multi-agent architectures. Without rigorous guardrails, multi-agent pipelines succumb to hallucination feedback loops and code drift. The AI Unified Process leverages the specification artifact chain as the shared immutable state substrate across agent topologies: 1) Spec Review Agent: Executes deterministic static analysis and advisory reviews on `UC-XXX.md` artifacts. 2) Code Implementation Agent: Ingests approved specs and stack plugins to synthesize domain services and presentation controllers. 3) Test Generation Agent: Reads failure postconditions and business rules to synthesize orthogonal test suites. Because the test agent works independently from the specification rather than inspecting generated production code, it prevents the classic bias of AI tests confirming AI mistakes. 4) Verification Agent: Runs test suites within sandboxed CI environments, providing deterministic feedback loops.

### ⚡ Modern Java 25 & Clean Architecture Uygulaması

### ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Çoklu Ajan Koordinasyon Arayüzü (Agent Orchestration Substrate)
package com.maang.aiup.multiagent;

public class MultiAgentOrchestratorDemo {
    public record SpecArtifact(String id, String content, boolean isApproved) {}
    public record GeneratedCode(String sourceFile, String testFile) {}

    // Ajan 1: Şartname Denetleyicisi
    public static class SpecLinterAgent {
        public boolean verify(SpecArtifact spec) {
            return spec.isApproved() && !spec.content().contains("SQL");
        }
    }

    // Ajan 2: Kod Üreticisi
    public static class CodeGeneratorAgent {
        public GeneratedCode generate(SpecArtifact spec) {
            return new GeneratedCode("LoanService.java", "LoanServiceTest.java");
        }
    }

    void main() {
        var spec = new SpecArtifact("UC-002", "System validates that copy is available (BR-004).",
true);
        var linter = new SpecLinterAgent();
        var generator = new CodeGeneratorAgent();

        if (linter.verify(spec)) {
            var code = generator.generate(spec);
            System.out.println("🤖 Çoklu Ajan İş Akışı Başarılı!");
            System.out.println("   -> Üretilen Kod: " + code.sourceFile());
            System.out.println("   -> Üretilen Bağımsız Test: " + code.testFile());
        }
    }
}
```

### ⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

### ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

#### 🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Kod üreten ajan ile test üreten ajanı aynı prompt zincirinde çalıştırmayın! Test ajanı koda bakarak değil, sadece şartnameye bakarak test yazmalıdır.

#### 🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never permit the test-generation agent to inspect generated production source code. Tests must be synthesized strictly from the specification to prevent self-affirming hallucinations.

### 🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Şartname, çoklu yapay zekâ ajanlarının etrafında toplandığı tek ortak gerçektir.

• Kod üreten ajan ile test üreten ajan birbirinden bağımsız çalışarak tarafsızlığı korur.

• Ajanlar arası iletişim serbest sohbetle değil, yapısal şartname zinciriyle yönetilir.

### 🎯 Key Architectural Takeaways

• The specification serves as the shared coordination substrate across autonomous AI agents.

• Code and test generation agents operate orthogonally to eliminate confirmation bias.

• Inter-agent workflows are governed by the structured artifact chain, not conversational chat.